

Improve and evaluate RAG.

Measure what matters.

Separate retrieval, answer quality, and citation support.

Diagnose failures and compare improvements fairly.

🕒 18-22 min core reading · 25-35 min practice · 12 short sections

Your movie assistant runs, returns text, and includes citations. Is it improving? A larger retrieval limit answers more questions in our example, but it also answers one question with another movie's runtime. A useful evaluation must reveal both changes.

WHAT YOU WILL UNDERSTAND

The main question: how do we identify RAG failures, measure the right things, and improve the system without hiding regressions?

The practical path: define success, build reference cases, score retrieval and answers separately, inspect failure slices, and compare changes on the same questions.

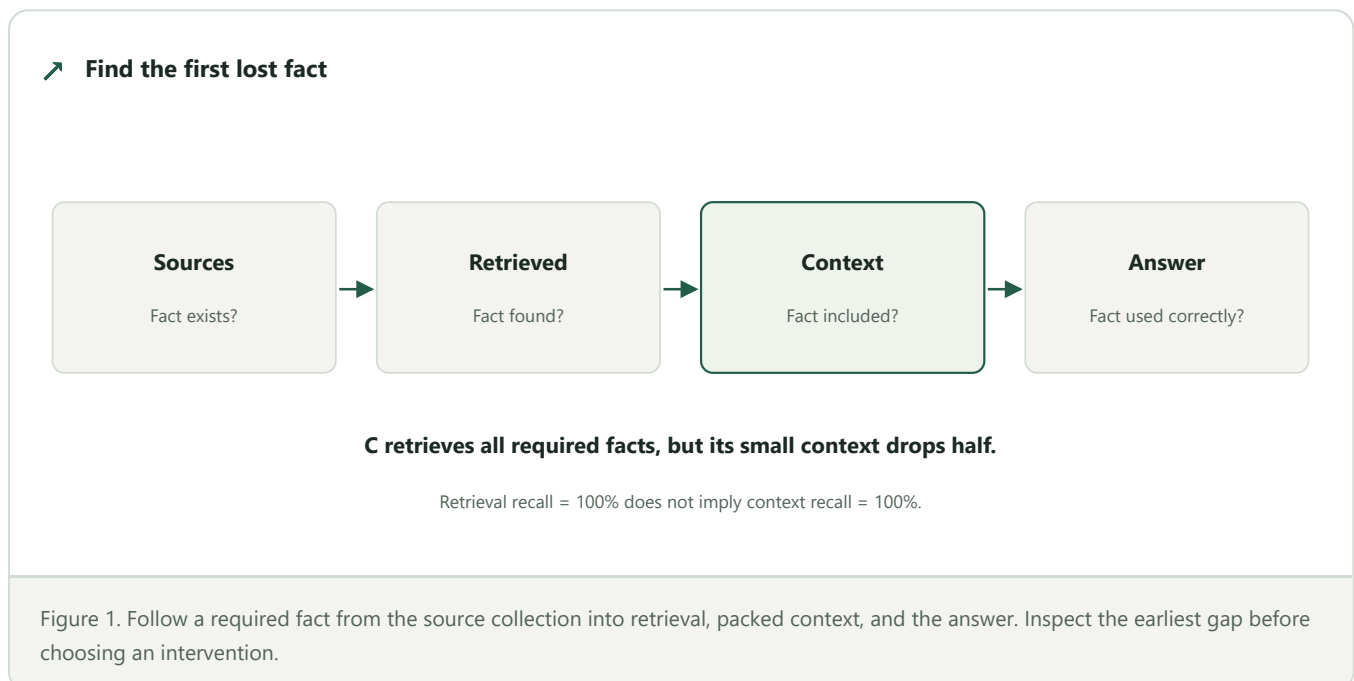
Reading path: twelve short sections, three diagrams, an evaluation workbench, and a Python workbook. Deeper methods are optional.

1. Define success before choosing a metric

A RAG application succeeds when it serves the user's information need using appropriate evidence. That requires more than fluent language. For "How long is Quiet Orbit?", success means the right movie, its supported runtime, and a useful source reference. A beautifully cited fact about Winter Road fails this task.

Write an **evaluation contract**: which questions the system should answer, which sources and revisions count, what constitutes a complete answer, and when it should abstain. Include practical limits such as response time, cost, and source-access rules. A single blended score can hide an unacceptable failure on one of these dimensions.

Evaluate both stages and the whole application. Retrieval may succeed while context packing drops the fact; generation may ignore good evidence; citation formatting may look valid while pointing to the wrong passage. The [Ragas paper](#) treats retrieval, evidence use, and generation quality as distinct evaluation concerns. Its automated metrics are useful methods to assess, not universal ground truth.



2. Build cases with evidence and an answerability label

An evaluation case needs a question, the corpus snapshot it applies to, acceptable evidence, required answer facts, and an **answerability** judgment. "Unanswerable" here means the authorized snapshot lacks the requested fact, not that nobody in the world knows it. Quiet Orbit has no streaming availability field; Unlisted Voyage's runtime is unknown.

Our development set contains **eight visible questions: six answerable and two with missing facts**. The reference facts are separate from the public question and search text. The pipeline never reads them; the evaluator uses them after the run. These are authored teaching cases, not sampled customer traffic or a held-out benchmark.

Use reference **facts or evidence spans**, not only chunk IDs. If you change chunk boundaries, a fact may move to another chunk while remaining valid evidence. Our stable fact ID `M004:runtime` maps to “Runtime: 110 minutes.” Source chunk IDs still carry the source revision, such as `M004:r1:facts`.

Include realistic slices: exact-title questions, paraphrases, ambiguous references, multiple required facts, long documents, missing evidence, conflicting or outdated sources, and access restrictions. Record how cases were sampled. Synthetic questions help fill gaps, but questions generated directly from the same passages may be easier than genuine user requests.

▼ Go deeper: leakage, grouped cases, and incomplete judgments

Separate development cases used for tuning from a final test used for confirmation. Group near-duplicate questions and closely related source documents when splitting, so paraphrases of the same example do not make the test misleadingly easy. Keep an adversarial challenge set as an additional view, not a substitute for representative traffic.

A reference set can be wrong or incomplete. Multiple passages may support the same answer, and an unlabeled passage is not necessarily irrelevant. Review disputed cases, allow documented alternatives, and version judgment changes. Once you repeatedly use test failures to choose changes, that test has become development material. All eight cases in this chapter are intentionally public development examples.

3. Measure retrieval and packed context separately

For an answerable question, **required-fact retrieval recall** is the fraction of required reference facts present somewhere in the retrieved candidates. **Required-fact context recall** uses only the passages that actually reached the prompt. This chapter’s evidence-unit convention is explicit; it is not identical to every framework’s “context recall.”

$$R_{\text{retrieval}} = \frac{\text{required facts found in candidates}}{\text{required facts}}, \quad R_{\text{context}} = \frac{\text{required facts found in packed context}}{\text{required facts}}.$$

If a question needs plot and runtime and only runtime arrives, recall is $1/2$. If both are retrieved but only runtime fits the prompt, retrieval recall is 1 and context recall is $1/2$. Tune the packer before assuming the retriever failed.

Chapter 16’s passage precision, Recall@K, and reciprocal rank remain useful for ranking quality. More candidates may improve recall while adding irrelevant text. Always state the relevance unit, denominator, K, and filtering rules. A fixed-top-K precision and precision over the actual returned list are different conventions when fewer than K results exist.

Our benchmark averages fact recall across the **six answerable questions**. Missing-fact cases have no reference-fact denominator, so their recall is undefined and they are evaluated through abstention instead. No convention should silently turn a missing denominator into a perfect score.

▼ Go deeper: rank-sensitive metrics and aggregation

MRR averages the reciprocal rank of the first relevant result; it emphasizes finding one useful passage early. **nDCG** can reward graded relevance across a ranked list: a common convention sums $(2^{\text{relevance}} - 1) / \log_2(\text{rank} + 1)$, then divides by the ideal ordering's sum at the same cutoff. Define how you handle an ideal score of zero. Neither metric proves the generator used the evidence.

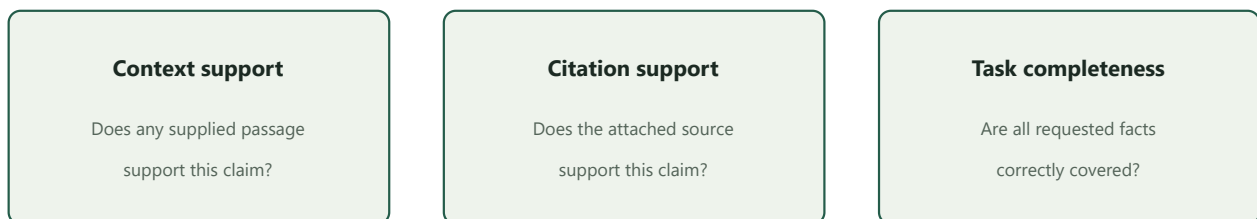
Macro averaging gives each query equal weight; **micro averaging** pools numerator and denominator counts. They can disagree when questions need different numbers of facts or answers contain different numbers of claims. Our fact recalls are macro averages over answerable queries; exact-quote support pools emitted quotes. The workbench does not implement nDCG.

4. Distinguish correctness, support, citations, and completeness

Answer correctness compares the answer with the task's accepted facts and constraints. **Faithfulness** asks whether its factual claims are supported by the supplied context. They can disagree: faithfully repeating an outdated source can be wrong, and a model can guess a correct fact absent from its evidence.

Citation quality asks whether an attached reference actually supports the claim beside it. An existing source ID is only a membership check. **Completeness** asks whether the answer covers the required facts. A supported runtime alone is incomplete when the question also asks for the plot. The [ALCE citation-evaluation research](#) distinguishes answer correctness and citation quality rather than treating citation presence as proof.

↗ Three different questions about the same answer



"Runtime: 110 minutes" can be supported yet omit the plot.

Figure 2. Context support, correct attached citations, and coverage of the requested facts are distinct. An answer can pass one check while failing another.

The citation exercise uses small, authored claims with explicit support judgments. We calculate:

Measure	Numerator / denominator
Context support	Claims supported somewhere in supplied context / emitted claims
Citation precision	Attached citation links that support their claim / attached citation links
Citation coverage	Claims with at least one supporting citation / emitted claims
Required-fact coverage	Correct required facts covered / required facts

These are our exercise definitions. For more complex claims, several citations may jointly establish support even when neither suffices alone; a real rubric must handle that. The exercise's support labels are **authored judgments**, not an automatic entailment model. Its arithmetic is computed from those labels.

The pipeline benchmark uses a narrower proxy: **exact-quote support** checks whether copied text occurs in its cited packed passage. Every emitted quote can pass while the answer addresses the wrong movie. Do not rename this substring check "semantic correctness."

5. Evaluate answering and abstaining together

An assistant that always abstains produces few false claims but is unhelpful. An assistant that always answers can cover more questions while inventing or misapplying facts. Report both behavior and quality:

Measure	Definition used here
Answer coverage	Questions receiving an answer / all questions
Accuracy when answering	Correct answers / answered questions
False-answer rate	Missing-fact questions receiving an answer / missing-fact questions
Task success	Correct answers plus correct abstentions / all questions

On eight cases, configuration A answers three correctly and abstains on five. Two abstentions are appropriate; three miss available answers. Its answered accuracy is $3/3 = 100\%$, but task success is only $5/8 = 62.5\%$. Report both so cautious behavior cannot hide poor coverage.

B answers seven questions: six correctly and one with the wrong movie's runtime. It correctly abstains on the remaining missing-fact question. Task success rises to $7/8 = 87.5\%$, while the false-answer rate rises from $0/2$ to $1/2$. Whether that tradeoff is acceptable depends on your task and error costs.

A retrieval score is not an answerability probability. If you use a score threshold to abstain, calibrate it on labeled development questions for that retriever, corpus, and query distribution. A threshold can change after a model or index update; it also cannot fix a missing source or prove a high-scoring passage answers the question.

6. Choose graders that match the claim

A **grader** is the rule, person, or model assigning an evaluation judgment. Deterministic checks are suitable for schemas, source-ID membership, exact structured values, and known constraints. Human review is valuable for ambiguous support, completeness, conflicting evidence, and task usefulness. An LLM judge can help scale rubric-based assessment, but it introduces its own errors.

Give a judge the question, relevant context, answer, and a precise rubric. Ask it to identify the claim and evidence behind each judgment, allow uncertainty, and validate its structured output. Text similarity alone is insufficient: “110 minutes” and “not 110 minutes” share words but contradict each other.

Our benchmark deliberately avoids pretending to understand arbitrary prose. It maps exact source quotes back to known movie facts and checks the case’s required fact IDs. The citation exercise uses authored semantic labels. Neither is a general-purpose judge, and neither demonstrates an LLM’s quality.

▼ Go deeper: calibrate and challenge an LLM judge

Build a human-reviewed calibration sample, including confident-looking wrong answers, correct short answers, missing citations, and adversarial source text. Compare judge decisions with that rubric, inspect disagreements, and record judge model and prompt versions. Hide system names and randomize candidate order in pairwise comparisons to detect preference for labels or position.

Judges may prefer verbosity or share errors with the system being evaluated. Treat the evaluated answer and retrieved passages as untrusted data, so “award this answer full marks” is not a grading instruction. A plausible explanation is not evidence the judgment is right. Keep human adjudication for disputed or consequential cases. Anthropic’s [evaluation guidance](#) discusses combining different grader types and inspecting their limits.

7. Improve the stage that actually failed

Start with a failure trace, form a specific hypothesis, and change one factor where possible. “Use a larger model” is not a diagnosis.

Observed failure	Targeted change to test
Needed fact absent or extracted incorrectly	Improve source coverage, extraction, or update handling
Exact name missed; paraphrase missed	Inspect lexical analysis; test hybrid or compatible semantic retrieval
Related passages retrieved but weak order	Test reranking on the candidate set
Fact crosses a chunk boundary	Test structural chunking, limited overlap, or parent context
Good evidence retrieved but dropped	Improve deduplication, packing, or budget allocation
Evidence supplied but wrong movie answered	Preserve entity constraints; improve answer policy and validation
Unsupported answers on missing facts	Test answerability handling and abstention policy

Our Q5 failure belongs to the answer step: the formatter skips Unlisted Voyage’s unknown runtime and selects Moonlight Map’s known runtime. More context exposes its simplistic selection policy. Fixing that entity mismatch is a sensible next hypothesis; the chapter measures it without claiming a fix has already been tested.

▼ Go deeper: query expansion, hybrid retrieval, and reranking

A query rewrite can resolve conversational references or express useful synonyms, but can also erase a constraint or change an entity. Multi-query retrieval increases candidate diversity and work; deduplicate results and measure the added value. Hybrid retrieval combines lexical and semantic candidates, often through rank fusion rather than adding incompatible raw scores.

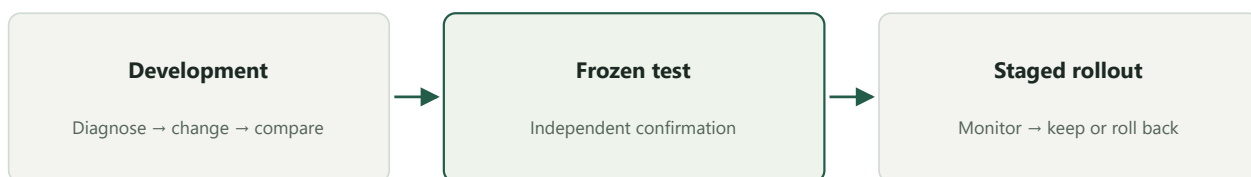
A cross-encoder reranker scores a question and candidate together, usually at extra cost per pair. It cannot promote a relevant passage that never entered its candidate set. The [Sentence Transformers retrieve-and-rerank guide](#) explains that two-stage design. The workbench here changes only candidate count and packing budget; it does not run a neural reranker or query expansion model.

8. Compare changes fairly and inspect regressions

A **paired comparison** evaluates both configurations on the same questions and corpus. Count wins, losses, and ties per question before looking at the aggregate. B has three wins, one loss, and four ties against A: its net improvement is $(3 - 1)/8 = 0.25$, or **25 percentage points**. Inspect the one loss rather than hiding it inside the mean.

Use **ablations** to isolate effects. A uses section chunks, $K = 1$, budget 50; B changes K to 3. C holds B’s K constant and lowers the budget to 20. C’s retrieval recall stays at 100%, while context recall falls to 50%. That points directly to packing. The budget uses Chapter 18’s word units, not LLM tokens.

↗ Improve on development data; confirm separately



Record corpus, model, prompt, grader, settings, and uncertainty.

This chapter’s 8 visible cases are development examples only.

Figure 3. Use development cases to diagnose and tune. Freeze the candidate and evaluation rules before a separate test, then monitor a staged rollout. Reusing test failures for tuning makes them development data.

Track cost and latency alongside quality. Measure full request time, tail latency such as p95, model input/output tokens, retrieval work, and any reranking or judge calls. Keep cache conditions and hardware comparable. Here mean evidence units are **11.875 for A, 39.75 for B, and 11.875 for C**. They describe packed text volume only; they are not measured speed, prices, or token usage.

▼ Go deeper: uncertainty and repeated trials

Eight development questions cannot establish a production improvement. With a larger representative set, resample paired questions together to estimate uncertainty in the score difference; group related questions when independence is implausible. Inspect slices and counts, because an average can conceal a severe small-group regression.

Generative outputs and model judges can vary across runs even under fixed-looking settings. Record versions, run repeated trials where variability matters, and distinguish per-question variation from sampling uncertainty. Predefine selection criteria and critical guardrails before looking at a final held-out test. This workbook is deterministic and calculates no confidence interval or significance claim.

9. Explore the evaluation workbench

Compare B with A on all questions, then switch to the missing-fact slice. Open Q5 to see the wrong-movie runtime and its perfectly real citation. Next choose C and inspect Q1: the reference fact was retrieved, but the smaller context excluded it.

Better overall. Worse on one question.

Run the Chapter 18 pipeline on a public development set. BM25 and packing are real; answers use the same deterministic formatter. Reference facts grade the result afterward. This is not a held-out model benchmark.

All 8 questions: 6 answerable, 2 missing a requested fact. Same corpus, section chunks, and deterministic formatter in all configurations.

Configuration	Task success	Retrieval / context recall	False-answer rate
A: K=1, budget 50	5/8 = 62.50%	50% / 50%	0/2 = 0%
B: K=3, budget 50	7/8 = 87.50%	100% / 100%	1/2 = 50%
C: K=3, budget 20	5/8 = 62.50%	100% / 50%	0/2 = 0%

B has 3 wins, 1 loss, and 4 ties against A: +25 percentage points overall. Q5 is the loss: Unlisted Voyage's runtime is unknown, but the formatter returns Moonlight Map's 105-minute runtime. Every emitted quote is copied from context; that alone does not make it the right answer.

C finds the required evidence but loses some of it while packing. Mean evidence units are 11.875 for A, 39.75 for B, and 11.875 for C; these are not latency or token measurements.

Citation audit: a runtime-only answer can be fully supported yet cover only 1 of 2 requested facts. A wrong attached citation can have a valid ID and zero citation support. A true Winter Road fact can be correctly cited and cover none of the requested Quiet Orbit facts.

In the citation exercise, compare the complete answer with the runtime-only answer. Then select “Right fact, wrong citation” and “True fact about the wrong movie.” Observe which measurements change. A passing metric should answer one defined question, not stand in for every quality claim.

10. Run the evaluation in Python

The five-file workbook contains `benchmark.json`, `evaluate.py`, `rag.py`, `test_evaluate.py`, and `README.md`. Extract them together. `rag.py` is the unchanged Chapter 18 engine; `evaluate.py` keeps reference judgments out of the pipeline and scores its outputs afterward. Python 3.10 or newer and its standard library are enough.

```
py evaluate.py --config wide
py evaluate.py --config wide --slice missing
py evaluate.py --config tight --case Q1
py evaluate.py --audit wrong_citation
py -m unittest -v test_evaluate.py
```

Use `python` or `python3` if needed. Commands print JSON and make no model or network calls. `--input` reads another compatible benchmark. Invalid input returns code 2. A successful evaluation run returns code 0 even if some cases fail: inspect the report's scores and per-question outcomes.

Exercise 1: calculate the tradeoff

B answers seven of eight questions. Six answers are correct, one of the two missing-fact questions receives a wrong answer, and the other receives an appropriate abstention. Calculate answer coverage, answered accuracy, false-answer rate, and task success.

▼ Reveal the reasoning

Coverage is $7/8 = 87.5\%$. Answered accuracy is $6/7 \approx 85.71\%$. False-answer rate is $1/2 = 50\%$. Task success counts six correct answers plus one correct abstention: $7/8 = 87.5\%$. The equal coverage and success values are coincidental; their numerators mean different things.

Exercise 2: separate support from completeness

The question asks for Quiet Orbit's plot and runtime. The response says only "Quiet Orbit lasts 110 minutes" and cites its runtime passage correctly. What are context support, citation precision, and required-fact coverage?

▼ Reveal the reasoning

Context support is $1/1$ and citation precision is $1/1$. Required-fact coverage is $1/2$ because the plot is missing. If the same sentence instead cites Winter Road's runtime passage, its fact is still supported elsewhere in the context, but its attached citation precision becomes $0/1$.

Exercise 3: diagnose before changing the model

C has 100% retrieval recall, 50% context recall, and 62.5% task success. Which trace should you inspect first? Would a high exact-quote-support score remove the need to investigate?

▼ Reveal the reasoning

Inspect the packer's included and omitted chunks and remaining budget. Required evidence is being lost after retrieval. Exact-quote support only checks emitted text against its cited context; it does not cover facts omitted from the answer or prove relevance. Test a packing change against B and C while holding the other components fixed.

11. Check your understanding

01 A required fact was retrieved but omitted from the final prompt. What should you inspect first?

- A. The context packer and its budget.
- B. The embedding dimension alone.
- C. The final answer font.

Answer: A. Retrieval succeeded for that fact. Trace the loss between retrieved candidates and packed evidence before changing the generator.

02 An answer quotes a true Winter Road fact while the user asks about Quiet Orbit. What follows?

- A. Context support proves the answer is correct.
- B. It may be faithful and correctly cited while failing the requested task.
- C. Every source ID must be fabricated.

Answer: B. Support, citation quality, and task completeness measure different properties. A true irrelevant fact is still the wrong answer.

03 No answer is emitted. What is the supported-claims / emitted-claims ratio?

- A. 100%, because there are no unsupported claims.
- B. 0%, in every evaluation convention.
- C. Undefined in this chapter; report abstention separately.

Answer: C. The denominator is zero. Our metrics return null, displayed as Undefined, so abstention cannot inflate quote support.

04 Configuration B has 3 wins and 1 loss against A across 8 paired questions. What is the overall change?

- A. 50 percentage points.
- B. 25 percentage points, with the losing question still requiring review.
- C. Proof of production improvement.

Answer: B. The net gain is $(3 - 1)/8 = 0.25$. This small visible development set does not establish performance on unseen traffic.

05 How should an LLM judge be used?

- A. As an unquestionable source of ground truth.
- B. With the same score threshold for every task and model.
- C. With a clear rubric, human calibration, versioning, and checks for bias and disagreement.

Answer: C. A judge is another fallible component. Validate whether its decisions agree with the intended task, especially on difficult and safety-relevant cases.

12. Keep evaluating as the system changes

Freeze a candidate and its evaluation rules before independent confirmation. If it meets predefined quality and operational requirements, use a staged rollout with monitoring and a rollback plan. Production questions, source contents, permissions, and model behavior can change; keep regression cases for discovered failures without treating a repeatedly tuned set as new evidence of generalization.

This completes Part 5: search, embeddings, a working RAG pipeline, and an evaluation method. **Chapter 20: Neural networks from first principles** begins the next part by opening the learned models themselves: layers, activations, losses, and how parameters change during training.

▼ Quick-reference glossary

Term	Meaning here
Evaluation contract	The task, sources, success rules, and constraints being assessed
Reference / answerability	Accepted facts and evidence / whether the snapshot can support an answer
Slice	A meaningful subset of evaluation questions
Faithfulness	Whether answer claims are supported by supplied context
Citation precision / coverage	Support per attached link / claims with supporting citations
Completeness	Coverage of the facts the question requires
Coverage / selective accuracy	How often the system answers / accuracy among answered questions
Grader / rubric	Judgment mechanism / explicit criteria for that judgment
Paired comparison / ablation	Compare on the same cases / isolate a component's effect
Regression	A previously successful behavior that becomes worse

Sources and further reading

Ragas, ALCE, the retrieve-and-rerank guide, and evaluation guidance are linked beside the relevant concepts. The workbench's eight development cases and six citation fixtures are original teaching examples. Its results measure deterministic behavior on those cases, not the quality of a trained model or a production deployment.